

Dokumentacja Funkcjonalna Projektu: Generowanie Grafów Planarnych

Aleksandra Rybalko, Nella Karczewicz

14 kwietnia 2026

1 Cel projektu

1.1 Cel główny

Celem projektu jest implementacja programu w języku C, który automatycznie wyznacza współrzędne wierzchołków grafu planarnego w celu stworzenia jego przejrzystej i estetycznej wizualizacji. Aplikacja przyjmuje opis krawędzi grafu, a następnie, wykorzystując algorytmy **Fruchterman-Reingolda** oraz **Tutte’go**, oblicza pozycje x i y dla każdego węzła, dążąc do minimalizacji przecięć krawędzi. Następnie zapisuje wynik działania programu jako plik tekstowy lub binarny.

1.2 Podcele

Cele szczegółowe projektu obejmują następujące zadania:

- Wczytanie grafu z pliku tekstowego.
- Wyznaczanie współrzędnych węzłów dla przystępnej i czytelnej wizualizacji grafu planarnego przedstawionego w postaci listy krawędzi poprzez implementację dwóch algorytmów: Fruchterman-Reingolda oraz Tutte.
- Zapisanie wyniku działania programu jako pliku tekstowego lub binarnego (według wyboru użytkownika) zawierającego listę współrzędnych wierzchołków.

1.3 Wymagania techniczne

Program został zaprojektowany zgodnie z poniższymi wymaganiami technicznymi:

- Kod musi być zgodny z dobrymi praktykami programowania w języku C oraz powinien być elastyczny i przejrzysty.
- Program powinien kompilować się przy użyciu kompilatora GCC.
- Aplikacja w języku C jest sterowana jedynie z linii poleceń, przez argumenty i opcje.
- Program wczytuje dane w formacie:
`<nazwa_krawędzi> <wierzchołek_A> <wierzchołek_B> <waga_krawędzi>`

Przykład wejścia:

```
AB;1;2;1
BC;2;3;1
CD;3;4;1
DB;4;2;1.407
```

Przykład wyjścia: Wynikiem działania jest plik (txt lub bin) o strukturze:
<wierzchołek> <współrzędna_x> <współrzędna_y>

```
1;0.0;0.0
2;1.0;0.0
3;1.0;1.0
4;0.0;1.0
```

2 Ogólne funkcje programu

Aplikacja odpowiada za wykonywanie następujących zadań:

1. **Wczytywanie struktury grafu:** Program odczytuje dane wejściowe jako pliki tekstowe zawierające listę krawędzi wraz z ich wagami i buduje wewnętrzną reprezentację powiązań między węzłami.
2. **Automatyczne wyznaczanie współrzędnych:** Główna funkcja programu polegająca na zamianie listy połączeń w konkretne punkty na płaszczyźnie. Program dąży do optymalnego rozłożenia wierzchołków, minimalizując ich nakładanie się na siebie.
3. **Wybór algorytmu:** Użytkownik ma możliwość wskazania, który algorytm (Fruchterman-Reingolda lub Tutte) ma zostać użyty do obliczeń poprzez parametry wywołania.
4. **Zarządzanie formatami danych:** Konwersja danych wejściowych na ustandaryzowany format wyjściowy, umożliwiającą późniejszą prezentację wyników w aplikacjach graficznych (np. w module Java).

3 Specyfikacja flag sterujących (CLI)

Program sterowany jest za pomocą argumentów przekazywanych w wierszu poleceń. Poniższa tabela przedstawia dopuszczalne flagi oraz zakresy ich parametrów:

3.1 Przykłady wywołania programu

Obliczenia standardowe (format tekstowy):

```
./graph_layout -i graf.txt -o wynik.txt -alg 0 -n 1000
```

Obliczenia zoptymalizowane (format binarny):

```
./graph_layout -i graf.txt -o punkty.bin -alg 1 -f bin
```

Flaga	Argument	Wartość	Opis funkcjonalny
-i	<path>	Ścieżka	Plik wejściowy. Lokalizacja pliku tekstowego z listą krawędzi.
-o	<path>	Ścieżka	Plik wyjściowy. Nazwa i lokalizacja pliku wynikowego.
-alg	<id>	0 lub 1	Wybór algorytmu. Pozwala na zmianę metody obliczeniowej.
-f	<format>	txt lub bin	Format zapisu. txt : tekstowy, bin : binarny.
-n	<int>	> 0	Liczba iteracji. Precyzja procesu optymalizacji (domyślnie 500).
-h	brak	brak	Pomoc. Wyświetla instrukcję obsługi w konsoli.

Tabela 1: Zestawienie parametrów wiersza poleceń.

3.2 Walidacja argumentów i danych wejściowych

Program przed rozpoczęciem fazy obliczeniowej przeprowadza weryfikację poprawności:

- **Kontrola obecności plików:** Program sprawdza, czy plik wejściowy podany po flagie -i fizycznie istnieje na dysku.
- **Weryfikacja typów:** W przypadku podania wartości nieliczbowej dla parametru -n lub -alg, program przerywa działanie, wskazując błędny argument.
- **Spójność danych:** Program weryfikuje, czy każda linia pliku wejściowego zawiera wymagane cztery kolumny danych.

4 Specyfikacja formatów danych

4.1 Format wejściowy (Import)

Program obsługuje import danych w formacie tekstowym opartym na strukturze **CSV/SSV**. Każda linia pliku wejściowego definiuje pojedynczą krawędź grafu i musi składać się z czterech kolumn:

Nazwa_krawędzi; Wierzchołek_A; Wierzchołek_B; Waga

Szczegółowa walidacja pól i ograniczenia: W celu zapewnienia stabilności fazy obliczeniowej, nałożono następujące restrykcje na dane wejściowe:

- **Nazwa krawędzi (string):**
 - Maksymalna długość: **10 znaków** (optymalizacja pod limit 1000 krawędzi).
 - Dopuszczalne znaki: wyłącznie małe litery alfabetu łacińskiego (**a-z**) oraz cyfry.
 - Zabronione: spacje, wielkie litery, znaki specjalne oraz polskie znaki diakrytyczne (**ą, ę, ł** itd.).
 - Format: ciągły ciąg znaków bez przerw.
- **Identyfikatory wierzchołków (int):**
 - Typ: liczba całkowita dodatnia.

- Maksymalna długość: **4 cyfry** (zakres do 9999, przy limicie 1000 unikalnych węzłów).
- Zabronione: litery oraz znaki specjalne.
- **Waga krawędzi (double):**
 - Format: liczba dziesiętna z kropką jako separatorem.
 - Maksymalna długość zapisu: **4 znaki** (np. 1.00).
 - Autokorekta: program automatycznie zamienia przecinek na kropkę przed procesem parsowania.

Zasady interpretacji separatorów i liczb: Program stosuje elastyczne podejście do formatowania technicznego:

- **Separatory danych:** Jako rozdzielacz kolumn akceptowany jest średnik (;) lub przecinek (,).
- **Białe znaki:** Dowolna liczba spacji i tabulatorów między kolumnami jest ignorowana.
- **Standard EOL:** Obsługiwany jest standard LF (\n) typowy dla systemów Unix/Linux.

Przykład poprawnego rekordu: krawedz12;102;105;1.50

4.2 Format wyjściowy (Eksport)

Program generuje współrzędne wierzchołków w wybranym przez użytkownika formacie (txt lub bin).

4.2.1 Format tekstowy (TXT)

Dane wyjściowe są zapisywane w formacie kolumnowym, gdzie separatorem jest pojedyncza spacja. Każda linia reprezentuje docelowe położenie jednego węzła:

ID_Wierzchołka Współrzędna_X Współrzędna_Y

Przykład pliku wynikowego:

1;15.50;40.25

2;10.00;5.12

3;60.15;12.80

4.2.2 Format binarny (BIN)

W przypadku wyboru formatu binarnego (flaga -f bin), dane zapisywane są bez separatorów tekstowych w celu optymalizacji rozmiaru pliku i szybkości odczytu przez moduł wizualizacji. Struktura rekordu binarnego:

- **int** (4 bajty) – ID wierzchołka.
- **double** (8 bajtów) – Współrzędna X.
- **double** (8 bajtów) – Współrzędna Y.

4.3 Obsługa formatów niestandardowych

Program został wyposażony w mechanizm **ignorowania pustych linii** oraz linii zaczynających się od znaku hash (#), co pozwala na dodawanie komentarzy wewnątrz plików z danymi grafu bez wpływu na proces obliczeniowy.

5 Sytuacje wyjątkowe i błędy

Program został zaprojektowany z myślą o odporności na błędne dane wejściowe oraz nieprawidłowe parametry wywołania. W przypadku wystąpienia sytuacji krytycznej, aplikacja przerywa działanie, wypisuje komunikat o błędzie na standardowe wyjście błędów (`stderr`) i zwraca odpowiedni kod powrotu do systemu operacyjnego.

5.1 Tabela kodów błędów

Poniższa tabela zawiera zestawienie komunikatów, ich przyczyn oraz przypisanych im kodów wyjścia:

Komunikat błędu (<code>stderr</code>)	Znaczenie i przyczyna	Kod powrotu
Błąd: Nieprawidłowe flagi CLI	Brak flag <code>-i/-o</code> , brak wymaganych argumentów lub użyto nieznanego formatu wyjściowego.	1
Błąd: Krytyczny błąd odczytu	Błąd alokacji pamięci RAM podczas wczytywania lub niskopoziomowy problem z dostępem do pliku.	2
Błąd: Nie można otworzyć pliku	Plik wejściowy nie istnieje pod wskazaną ścieżką lub brak uprawnień do odczytu.	3
Błąd: Nieznany algorytm	Wybrano nazwę algorytmu inną niż dopuszczalne <i>tutte</i> lub <i>fruchterman</i> .	4
Błąd: Plik pusty	Wskazany plik wejściowy nie zawiera żadnych danych do przetworzenia.	5
Błąd: Graf zbyt mały	Graf posiada niewystarczającą liczbę elementów do poprawnego działania (wymagane $V \geq 3, E \geq 2$).	6
Błąd: Wynik nieplanarny	Wykryto przecięcia krawędzi po zakończeniu procesu optymalizacji układu.	7
Błąd: Graf niespójny	Test spójności DFS nie został zaliczony; graf składa się z niepołączonych składowych.	8
Błąd: Format wyjściowy	Przekazano nieprawidłowy format zapisu danych (wymagany <i>txt</i> lub <i>bin</i>).	9
Błąd: Struktura danych	Wykryto błędy w linii: pętle własne, niedozwolone znaki diakrytyczne lub zły format kolumn.	10

Tabela 2: Specyfikacja obsługi błędów i kodów wyjścia.

5.2 Sposób raportowania

Wszystkie błędy są raportowane w ustandaryzowany sposób, co ułatwia automatyzację pracy z programem (np. przy użyciu skryptów Bash lub modułu nadrzędnego w Javie). Przykładowy format raportowania:

```
[FATAL ERROR]: Błąd: Nieprawidłowy format danych w linii 42. Oczekiwano
wartości numerycznej.
```

5.3 Zabezpieczenia algorytmiczne

Program implementuje mechanizm „bezpiecznika” (*watchdog*), który przerywa obliczenia, jeśli po zadanej liczbie iteracji siły w modelu sprężynowym nie dążą do równowagi (zapobieganie pętlom nieskończonym przy specyficznych układach krawędzi).